Advanced Secure Messaging

AES-GCM · RSA-PSS Signatures · Multi-session · Audit Log

Denis O.

March 12, 2026

Contents

§ 1 Objective and Scope

Version 2 extends the baseline v1 system with production-grade security properties:

- **Authenticated encryption**: AES-CBC (unauthenticated) is replaced by AES-GCM, which simultaneously encrypts and provides a 128-bit integrity tag — preventing ciphertext tampering without a separate HMAC.
- **Non-repudiation**: every outgoing message is signed with the sender's RSA private key (PSS/SHA-256). Recipients verify the signature against the sender's registered public key.
- **Multi-session**: each client maintains a per-peer AES key dictionary instead of a single global variable, enabling simultaneous conversations.
- **Server-side signature verification**: the server verifies the session-request signature before issuing a new AES key, preventing impersonation.
- **Idempotent session management**: duplicate session requests return HTTP 409 rather than silently re-generating a key; a `/session/reset` endpoint allows intentional re-keying.
- **Structured messages**: message payloads carry `text`, `timestamp`, `signature`, `sender`, and `session_id` — enabling richer UI and auditability.
- **Audit log**: the server persists a timestamped log of all cryptographic events accessible via `GET /audit`.
- **Modern UI**: a dark-mode interface with message timestamps, RSA-PSS verification badges, encryption summary panel, and toast notifications.

§ 2 Architecture

2.1 Component overview

Process	Port	Role
`server.py`	5000	Key distribution, signature verification, audit log
`client.py` Alice	5001	Chat client, AES-GCM encrypt/decrypt, RSA-PSS sign/verify
`client.py` Bob	5002	Chat client (same code, different port/username)

2.2 Full message flow

1. Both clients generate RSA-2048 key pairs locally at startup.
2. Each client `POST /register` their public key with the server.
3. Alice calls `POST /session` with a PSS-signed proof-of-identity string `"Alice:Bob"`.
4. Server verifies the signature, generates a 16-byte AES key, encrypts it under each party's RSA public key, records the session in the audit log, and returns both ciphertexts.
5. Alice decrypts her copy; forwards Bob's RSA-wrapped copy (plus session ID) via peer-to-peer `POST /receive_key`.
6. Alice composes message payload $\{\texttt{text}, \texttt{timestamp}, \texttt{signature}, \texttt{session_id}\}$, signs the plaintext with RSA-PSS, serialises to JSON, and encrypts the whole JSON string with AES-GCM.
7. The AES-GCM blob is sent to Bob via `POST /receive_message`.
8. Bob decrypts with AES-GCM (tag verification happens automatically), parses the JSON, fetches Alice's public key, and verifies the RSA-PSS signature.
9. Bob's UI displays the message with a green badge if verification succeeds.

§ 3 Cryptographic Details

3.1 AES-GCM authenticated encryption

AES-128-GCM provides *confidentiality* (via CTR-mode encryption) and *integrity* (via GHASH authentication tag) in a single pass. The 96-bit nonce is generated randomly for each message and prepended to the tag+ciphertext blob before base64 encoding.

$$\text{blob} = \underbrace{\text{nonce}}_{16\,\text{B}} \parallel \underbrace{\text{tag}}_{16\,\text{B}} \parallel \underbrace{\text{ciphertext}}_{|\text{plaintext}|\,\text{B}}$$

```python
def encrypt_gcm(plaintext: str, key: bytes) -> str:
    cipher = AES.new(key, AES.MODE_GCM)
    ciphertext, tag = cipher.encrypt_and_digest(plaintext.encode
        ())
    blob = cipher.nonce + tag + ciphertext
    return base64.b64encode(blob).decode()
```

If the tag does not match on decryption, `decrypt_and_verify` raises `ValueError` — the message is rejected before any plaintext is exposed.

3.2 RSA-PSS message signatures

Each message is signed over its plaintext with the sender's RSA-2048 private key using the PSS scheme and SHA-256:

```
def sign_message(text: str) -> str:
    h   = SHA256.new(text.encode())
    sig = pss.new(my_key).sign(h)
    return base64.b64encode(sig).decode()
```

The signature is embedded in the JSON payload *before* encryption, so only the intended recipient (who decrypts successfully) can read and verify it.

3.3 Server-side session-request signature

Before issuing an AES session key, the server verifies that the claimed initiator signed the string `"initiator:target"`:

```
verifier = pss.new(RSA.import_key(clients_db[initiator]["
    public_key"]))
h        = SHA256.new(f"{initiator}:{target}".encode())
verifier.verify(h, base64.b64decode(sig_b64))    # raises on
    failure
```

This ensures the server does not issue session keys to an entity that cannot prove ownership of the registered private key.

3.4 Comparison: v1 vs v2

Property	v1	v2
Symmetric cipher	AES-128-CBC	AES-128-GCM
Integrity	None (unauthenticated)	128-bit GCM tag
Message signatures	None	RSA-2048 PSS/SHA-256
Session-req. authN	None	RSA-PSS proof-of-identity
Multi-session	No (global variable)	Yes (per-peer dict)
Idempotent sessions	No	Yes (HTTP 409 + reset)
Structured messages	Plain strings	JSON with timestamp + sig
Audit log	None	Timestamped server-side log
HTTP error codes	200 for errors	400 / 403 / 404 / 409
UI	Basic light theme	Dark theme + sig badges + status

§ 4 API Reference

4.1 Server endpoints

Method	Route	Description
GET	`/`	Health check + server stats
POST	`/register`	Register username + RSA public key
GET	`/users`	List all registered clients
POST	`/session`	Issue AES session key (with signature verification)
POST	`/session/reset`	Force-delete a session between two users
GET	`/session_info/<id>`	Return session metadata
GET	`/audit`	Return full audit event log

4.2 Client endpoints

Method	Route	Description
GET	`/`	Serve the v2 chat UI
POST	`/send`	Sign, encrypt (GCM), forward message
POST	`/receive_key`	Receive RSA-wrapped AES key
POST	`/receive_key_v2`	Extended: also caches sender's public key
POST	`/receive_message`	Decrypt (GCM), verify signature, store
GET	`/messages`	Return history (`?peer=X` to filter)
GET	`/status`	Expose active sessions + message counts
POST	`/reset_session`	Clear local AES key for a peer

§ 5 How to Run

```
# Terminal 1 - key server
python server.py

# Terminal 2 - Alice
python client.py 5001 Alice

# Terminal 3 - Bob
python client.py 5002 Bob
```

Open http://127.0.0.1:5001 and http://127.0.0.1:5002 in two browser tabs. The UI auto-configures the target peer.

To inspect the server audit log:

```
curl http://127.0.0.1:5000/audit
```

To view client session status:

```
curl http://127.0.0.1:5001/status
```

§ 6 Design Notes and Remaining Limitations

- **Server trust**: The server still sees the AES key in plaintext at generation time. A Diffie–Hellman or ECDH key agreement would eliminate this.
- **No TLS**: HTTP transport is unencrypted. Production deployment requires HTTPS.
- **Public key retrieval**: Signature verification relies on fetching the sender's public key from the server via `GET /users`. In a real system this should be done over a separate authenticated channel, or keys should be certified by a CA.
- **Replay attacks**: messages carry a timestamp but there is no nonce registry to detect replays. Adding a per-session message counter would address this.
- **In-memory storage**: all state is lost on restart. A real deployment would persist sessions and message history in a database.

§ 7 Possible Extensions

- Replace server AES distribution with an ECDH key agreement (zero server trust).
- Add TLS to all Flask endpoints (`ssl_context` in `app.run`).
- Implement a replay-prevention nonce registry.
- Persist sessions and history to SQLite.
- Group chat support (one AES key per group, distributed to n members).

§ 8 References

- NIST SP 800-38D — Galois/Counter Mode (GCM): https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38d.pdf
- PKCS #1 v2.2 / RSA-PSS — RFC 8017: https://datatracker.ietf.org/doc/html/rfc8017
- PyCryptodome documentation: https://pycryptodome.readthedocs.io
- Flask documentation: https://flask.palletsprojects.com